

Guide Complet : Python, uv, VS Code & Jupyter Notebook

Portfolio Technique / Recherche Scientifique

Année 2026

Table des matières

Pourquoi choisir uv + VS Code en 2026 ?	2
1 Prérequis et Installation d'uv	2
2 Initialisation du projet et de l'environnement	2
3 Étape Cachée : Autorisation et Activation de l'environnement	3
4 Installation des Bibliothèques (Data Science & Jupyter)	3
5 Configuration de Visual Studio Code	3
6 Création et Exécution du Notebook Hello World	4
7 Production : Exécuter l'application Data Science (app.py) avec uv run	4

Pourquoi choisir uv + VS Code en 2026 ?

Le choix de vos outils de développement a un impact direct sur votre productivité et votre carrière. Délaisser les installations globales ou les environnements lourds (comme Anaconda) pour le duo **uv** et **Visual Studio Code** offre des avantages majeurs :

- **Vitesse instantanée** : Écrit en Rust, **uv** est 10 à 100 fois plus rapide que **pip** et **conda** pour installer vos packages et résoudre les dépendances.
- **Légèreté et isolation** : **uv** télécharge et gère vos versions de Python de manière transparente, sans polluer votre système d'exploitation. Vos projets restent parfaitement isolés dans un dossier `.venv` local.
- **Flexibilité maximale** : **VS Code** est un éditeur de code ultra-léger, modulaire et multi-plateforme (Windows/macOS), qui s'interface nativement avec les environnements virtuels créés par **uv**.
- **Valorisation professionnelle et scientifique (GitHub)** : Le format Jupyter Notebook (`.ipynb`) intègre à la fois le code, les visualisations et les explications textuelles. Cette structure le rend **idéal pour être téléversé (uploadé) comme dépôt (repository) sur GitHub**. En publiant vos Notebooks de manière claire et documentée sur GitHub, vous construisez un portfolio concret qui :
 - **Propulse votre profil sur le marché du travail** en servant de CV technique visuel pour les recruteurs et les entreprises.
 - **Crédibilise vos travaux en recherche scientifique** en facilitant la répliquabilité de vos expériences, le partage Open Source et la collaboration entre pairs.

1 Prérequis et Installation d'uv

Ouvrez votre terminal (PowerShell sur Windows ou Terminal sur macOS) :

Sur Windows (PowerShell) :

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh | iex"
```

Sur macOS / Linux (Terminal) :

```
curl -LsSf https://astral.sh | sh
```

Important

Fermez et rouvrez complètement votre terminal après l'installation pour que la commande **uv** soit reconnue par votre système.

2 Initialisation du projet et de l'environnement

Placez-vous dans votre espace de travail pour générer la structure de votre projet.

```
# 1. Créer et entrer dans le dossier du projet
mkdir python_uv_workspace && cd python_uv_workspace

# 2. Initialiser le projet (génère un fichier pyproject.toml)
uv init
```

```
# 3. Créer explicitement l'environnement virtuel local
uv venv
```

3 Étape Cachée : Autorisation et Activation de l'environnement

Pour installer ou exécuter des outils directement depuis le terminal, vous devez activer votre environnement virtuel.

Sur Windows (PowerShell) :

Par défaut, Windows bloque l'exécution des scripts par sécurité. Vous devez d'abord autoriser la session en cours avant d'activer l'environnement :

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process
.venv\Scripts\activate
```

Sur macOS / Linux (Terminal) :

```
source .venv/bin/activate
```

Le terminal affiche désormais le préfixe (.venv) indiquant que l'isolement est actif.

4 Installation des Bibliothèques (Data Science & Jupyter)

Nous installons le moteur de Notebook ainsi qu'une suite essentielle pour l'analyse et la visualisation de données (*pandas*, *matplotlib*, *seaborn*) en une seule commande :

```
uv add ipykernel pandas matplotlib seaborn
```

Étape cachée de secours (Enregistrement du Kernel) :

Si VS Code peine à détecter automatiquement votre environnement virtuel, forcez son enregistrement dans le système pendant que votre environnement est activé :

```
python -m ipykernel install --user --name=python_uv_workspace --display-name "Python (uv-env)"
```

5 Configuration de Visual Studio Code

1. Lancez votre éditeur de code dans le dossier actuel : `code .`
2. Ouvrez l'onglet **Extensions** (Ctrl+Shift+X ou Cmd+Shift+X).
3. Installez les trois extensions officielles indispensables :
 - **Python** (par Microsoft)
 - **Jupyter** (par Microsoft)
 - **Code Runner** (par FormulaHendry — *optionnel*)

6 Création et Exécution du Notebook Hello World

1. **Créer le Notebook** : Dans l'explorateur de VS Code, créez un fichier nommé `hello.ipynb`.
2. **Associer le Kernel** : En haut à droite du Notebook, cliquez sur **Select Kernel**, choisissez **Python Environments...** puis sélectionnez la ligne `.venv` ou **Python (uv-env)**.
3. **Écrire le code** : Dans la première cellule, copiez :

```
print("Hello World avec uv et VS Code !")
```

4. **Exécuter** : Appuyez sur **Maj + Entrée**. Le texte s'affiche instantanément en dessous.

7 Production : Exécuter l'application Data Science (app.py) avec uv run

Isolez la logique d'analyse de données dans un fichier de script Python standard (`app.py`).

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Test rapide des bibliothèques installées
data = {'Catégories': ['A', 'B', 'C', 'D'], 'Valeurs': 1, 2, 3, 4}
df = pd.DataFrame(data)

# 2. Génération d'un graphique de validation
sns.set_theme(style="darkgrid")
sns.barplot(x='Catégories', y='Valeurs', data=df)
plt.title("Validation de l'environnement uv")
plt.show()
```

Exécution simplifiée via le Terminal Intégré

Ouvrez le terminal intégré de VS Code et tapez simplement :

```
uv run app.py
```

Astuce d'automatisation (Bouton Play de VS Code)

Pour lier l'extension Code Runner à `uv run`, ouvrez votre fichier `settings.json` de VS Code et ajoutez :

```
"code-runner.executorMap": {
  "python": "uv run"
}
```